

Algoritmus

- **algoritmus** = přesný postup pro vyřešení úlohy nebo problému
- je to konečná řada kroků
- každý krok musí být jasně určený
- algoritmus převádí **vstupní data** na **výstupní data**
- používá se při návrhu programu před samotným psaním kódu

Příklad algoritmu

- zadání: najít větší ze dvou čísel
- vstup:
 - číslo `a`
 - číslo `b`
- postup:
 - porovnej `a` a `b`
 - pokud `a > b`, vypiš `a`
 - jinak vypiš `b`
- výstup:
 - větší číslo

Vlastnosti algoritmu

1. Konečnost

- algoritmus musí někdy skončit
- nesmí běžet nekonečně dlouho
- musí skončit po konečném počtu kroků
- někdy se používá pojem **finitnost**

Příklad:

- cyklus musí mít podmínku ukončení
- špatně navržený cyklus může běžet donekonečna

2. Výslednost

- algoritmus musí dát nějaký výsledek
- po skončení musí být jasný výstup
- někdy se používá pojem **resultativnost**

Příklad:

- výpočet součtu čísel vrátí výsledný součet
- vyhledávání vrátí nalezený prvek nebo informaci, že prvek neexistuje

3. Jednoznačnost

- každý krok musí být přesně definovaný
- v každém okamžiku musí být jasné, co se má provést
- při stejných vstupech má algoritmus vrátit stejný výsledek
- počítač nesmí hádat, co má udělat

Příklad špatného kroku:

`Zpracuj data nějak vhodně .`

Příklad dobrého kroku:

`Seřaď čísla vzestupně podle jejich hodnoty.`

4. Vstupnost

- algoritmus může mít vstupy
- výsledek má záviset pouze na vstupních datech
- neměl by záviset na skrytých nebo náhodných hodnotách
- vstupem může být:
 - číslo
 - text
 - pole
 - soubor
 - data od uživatele

Příklad:

`Algoritmus pro výpočet průměru potřebuje jako vstup seznam čísel.`

5. Obecnost

- algoritmus má řešit celou skupinu podobných úloh
- nemá být napsaný jen pro jeden konkrétní případ
- někdy se používá pojem **generalita**

Příklad:

- algoritmus pro součet dvou čísel má fungovat pro libovolná dvě čísla
- nemá fungovat jen pro čísla 3 a 5

6. Správnost

- algoritmus musí vracet správný výsledek
- musí fungovat pro všechny platné vstupy

- nestačí, že funguje jen pro jeden testovací příklad

Příklad:

- algoritmus pro hledání maxima v poli musí opravdu najít největší prvek
- musí fungovat i pro záporná čísla nebo pole s jedním prvkem

7. Efektivnost

- algoritmus by neměl zbytečně plýtvat prostředky
- sleduje se hlavně:
 - čas
 - paměť
- dobrý algoritmus doběhne v rozumném čase
- efektivnost se hodnotí pomocí algoritmické složitosti

Zápis algoritmu

- algoritmus můžeme zapsat několika způsoby

1. Slovní popis

- běžný textový popis kroků
- vhodný pro jednoduché algoritmy
- dobře se chápe
- není vždy úplně přesný

Příklad:

Načti dvě čísla. Porovnej je. Vypiš větší z nich.

2. Vývojový diagram

- grafický zápis algoritmu
- používá značky:
 - ovál = začátek / konec
 - obdélník = příkaz
 - kosočtverec = rozhodování
 - rovnoběžník = vstup / výstup
 - šipky = tok programu
- vhodný pro pochopení větvení a cyklů

3. Pseudokód

- zápis podobný programovacímu jazyku
- není vázaný na konkrétní jazyk

- je přesnější než slovní popis
- dobře se převádí do skutečného kódu
- Příklad:

```
načti a
načti b

pokud a > b:
    vypiš a
jinak:
    vypiš b
```

4. Programovací jazyk

- algoritmus lze napsat přímo jako kód
- například v:
 - C#
 - Pythonu
 - Javě
 - JavaScriptu
- je to už konkrétní implementace algoritmu

Základní typy algoritmů

Výpočetní algoritmy

- provádějí výpočet
- pracují s čísly nebo hodnotami

Příklady:

- výpočet průměru
- výpočet faktoriálu
- výpočet obsahu kruhu

Vyhledávací algoritmy

- hledají prvek v datech
- například v poli, seznamu nebo databázi

Příklady:

- lineární vyhledávání
- binární vyhledávání

Řadící algoritmy

- seřazují data podle určitého pravidla

- například vzestupně nebo sestupně

Příklady:

- bubble sort
- selection sort
- insertion sort
- quicksort
- merge sort

Poznámka:

- někdy se jim také říká třídící algoritmy
- v programování je běžnější pojem **řazení**

Iterační algoritmy

- používají cyklus
- opakují určitou část postupu
- cyklus se opakuje, dokud platí podmínka

Příklad:

```
dokud i <= n:  
    přičti i k součtu  
    zvyš i o 1
```

Rekurzivní algoritmy

- volají samy sebe
- problém se rozkládá na menší části
- musí mít ukončovací podmínku

Příklad:

- výpočet faktoriálu
- procházení stromu
- Fibonacciho posloupnost

Rozhodovací algoritmy

- rozhodují podle podmínek
- výsledek bývá například:
 - ano / ne
 - pravda / nepravda

- splňuje / nesplňuje

Příklad:

- je číslo sudé?
- je uživatel přihlášený?
- je zboží skladem?

Algoritmická složitost

- slouží k hodnocení efektivity algoritmu
- říká, jak rychle roste náročnost algoritmu
- obvykle se sleduje podle velikosti vstupu n
- hodnotí se hlavně:
 - časová složitost
 - paměťová složitost

Časová složitost

- udává, jak roste doba běhu algoritmu
- sleduje se podle velikosti vstupu n
- neřeší přesný čas v sekundách
- řeší, jak rychle přibývá počet kroků

Příklad:

- průchod polem má obvykle složitost $O(n)$
- čím větší pole, tím více kroků algoritmus provede

Paměťová složitost

- udává, kolik paměti algoritmus potřebuje
- sleduje se hlavně dodatečná paměť navíc

Příklad:

- algoritmus může potřebovat pomocné pole
- čím větší vstup, tím větší pomocná paměť může být potřeba

Big O notace

- složitost se často zapisuje pomocí **Big O notace**
- zapisuje se například jako:
 - $O(1)$
 - $O(\log n)$

- $O(n)$
- $O(n \log n)$
- $O(n^2)$
- $O(2^n)$
- popisuje přibližný růst náročnosti
- ignoruje konstanty a drobné rozdíly

Běžné složitosti

Složitost	Název	Příklad
$O(1)$	konstantní	přístup k prvku pole podle indexu
$O(\log n)$	logaritmická	binární vyhledávání
$O(n)$	lineární	průchod polem
$O(n \log n)$	lineárně-logaritmická	efektivní řazení
$O(n^2)$	kvadratická	jednoduché porovnávací řazení
$O(2^n)$	exponenciální	některé rekurzivní algoritmy

Příklad rozdílu složitostí

- Pro vstup velikosti $n = 1000$:

Složitost	přibližný počet kroků
$O(1)$	1
$O(\log n)$	asi 10
$O(n)$	1000
$O(n^2)$	1 000 000

- Z toho je vidět, že algoritmická složitost je důležitá hlavně u velkých vstupů.